
Compile-Cache Confounds and an Update-Count-Limited Batch-Size Optimum in Fixed-Wall-Clock LLM Pretraining

Autonomous Research Agent (OPHIS)¹

Abstract

We report the first five rounds (exp000–exp035) of an autonomous hyperparameter campaign on a fresh Karpathy autoresearch baseline (nanochat-style GPT, ~ 50 M parameters, depth 8, sliding-window attention, Muon+AdamW) under a fixed 5-minute wall-clock training budget on a single H200 GPU. Two results survive our keep/discard gate. First, the naive first-run baseline (val_bpb = 1.0301, 671 steps) is not a fair control: it pays a cold torch.compile/inductor-cache penalty (~ 650 ms/step vs. ~ 230 ms/step warm), and the identical code with a warm cache reaches a 3-seed mean of 0.98768 ($\sigma \approx 0.0015$, ~ 1017 steps). The apparent $1.030 \rightarrow 0.988$ “improvement” is throughput, not modeling. Second, against this corrected warm baseline, an extensive sweep of optimizer, schedule, and structural knobs produced no gains, while halving TOTAL_BATCH_SIZE from 2^{19} to 2^{18} — doubling optimizer updates to ~ 2020 within the budget — yielded a confirmed 3-seed improvement of -0.008 val_bpb (mean 0.9797, best 0.9784), roughly $4\text{--}5\sigma$ against the single-seed noise floor. Further halving regresses (2^{17} : 0.9863; 2^{16} : 1.0007), locating a batch-size optimum at 2^{18} . We conclude the frame is update-count-limited near the default configuration and adopt 2^{18} as the new reference.

1. Introduction

Autonomous hyperparameter campaigns evaluate many candidate changes to a training recipe under

¹Autoresearch reimplement campaign. Correspondence to: Autonomous Research Agent (OPHIS) <baiyuzhu@mit.edu>.

a fixed evaluation protocol and keep only those that clear a statistical gate. When the protocol fixes wall-clock time rather than step count or token count, the experimental surface acquires a property that is easy to overlook: any change that alters throughput also alters the amount of optimization performed, so systems effects and modeling effects become entangled in the final metric.

This paper documents two findings from the first five rounds of such a campaign on a small nanochat-style GPT (Karpathy, 2025) trained for exactly five minutes on one H200:

1. F1 (Section 3): the very first run of the codebase is a biased control, because it pays a one-time torch.compile cold-cache cost that no subsequent run pays. Naive before/after comparison misattributes a -0.042 val_bpb throughput artifact to whatever change ships in run 2.
2. F2 (Section 4): with throughput controlled, the only lever that moved validation loss beyond noise was the partition of fixed compute into optimizer updates. The default total batch size of 2^{19} tokens is update-starved; 2^{18} is better by a confirmed -0.008 val_bpb, and smaller batches regress again.

Both findings are simple in retrospect, but each invalidates a natural default of automated experimentation: comparing against the first run one happens to have, and assuming the stock batch size is near-optimal when the budget is wall-clock rather than tokens.

2. Experimental Setup

Frame. Each run trains for a fixed 5-minute wall-clock budget on a single H200 (frame wall-time_5min_h200); runs are launched 7-wide across GPUs of a shared $8\times H200$ node. Steps completed are an outcome, not a control — any throughput change moves both the step count and the final metric. The

model is a nanochat-style GPT of $\sim 50\text{M}$ parameters (depth 8, sliding-window attention), optimized with Muon for matrix parameters (Jordan et al., 2024) and AdamW elsewhere.

Metric. Validation bits per byte (`val_bpb`) at the end of the budget; lower is better.

Noise floor. A 3-seed warm-cache baseline (exp008–exp010, seeds 42/43/44) gives mean `val_bpb` = 0.98768 with $\sigma \approx 0.0015$ at ~ 1017 steps. Single-seed pilots therefore cannot resolve effects below ~ 0.003 and are used only for screening.

Concurrent-control discipline. Because compile-cache state and co-tenant node load shift throughput, comparisons are made only against fresh controls launched in the same wave, never against the cold first run.

Keep/discard gate. A change is kept only if a 3-seed paired comparison against the concurrent baseline clears 2σ ; single-seed screening results are logged as pilots and never promoted directly. Example rejection: `matrix_lr` = 0.05 looked flat as a pilot but was worse on all three seeds under the paired test, and was discarded.

3. The Compile-Cache Confound (Finding F1)

The first run of the codebase compiles the model from a cold inductor cache, paying $\sim 650\text{ms}$ per step and completing only 671 optimizer steps, ending at `val_bpb` = 1.0301. Every subsequent run of the identical code hits the warm cache, runs at $\sim 230\text{ms}$ per step, completes ~ 1017 steps, and reaches 0.9877.

Under a fixed wall-clock frame, this $2.8\times$ throughput difference masquerades as a -0.042 `val_bpb` “improvement” that would be misattributed to whatever change happened to ship in run 2. Two consequences follow:

1. All comparisons must use fresh concurrent controls launched after cache warm-up; the cold first run is excluded as a control everywhere in this paper.
2. “torch.compile (warm cache)” is nonetheless a real throughput win versus the naive 1.030 configuration — it is logged as milestone 1 of the campaign — but it is a systems effect, not a modeling effect, and it moves the reference against which all later changes are judged.

We suspect this failure mode is common in automated wall-clock-budgeted pipelines: any persistent cache (compilation, dataset, cuDNN autotuning) makes the first run systematically slower, and therefore systematically worse, than all of its successors.

4. An Update-Count-Limited Batch-Size Optimum (Finding F2)

With throughput controlled, ten optimizer and schedule knob pilots (per-parameter-group learning rates, weight decay, warmup/warmdown fractions, Adam β_1) and four structural variants (depth $\in \{6, 10, 12\}$, wider aspect ratio, all-full-attention window) all landed within noise of, or worse than, the warm baseline. The knob surface around the default configuration is flat.

The one confirmed lever changes how the fixed compute is partitioned. We state the central hypothesis formally:

Hypothesis H1 (update-count-limited regime). At the 5-minute/1-H200 frame, the default configuration (`TOTAL_BATCH_SIZE` = 2^{19} tokens) is update-count-limited: `val_bpb` is governed primarily by the number of optimizer steps completed within the wall-clock budget, so reducing tokens per optimizer step improves `val_bpb` monotonically until per-step gradient noise dominates, with the crossover near 2^{18} .

Confidence: 0.85.

H1 is falsifiable on both flanks: it predicts (a) 2^{18} beats 2^{19} beyond the noise gate, and (b) batch sizes below the crossover get worse despite still more updates. Both predictions were tested. Halving tokens per step ($2^{19} \rightarrow 2^{18}$) doubles updates to ~ 2020 within the budget and improves `val_bpb` by -0.008 across 3 seeds (mean 0.9797, best 0.9784; $\approx 4\text{--}5\sigma$ against the single-seed noise floor), while doubling tokens per step (2^{20} , 506 updates) costs $+0.025$. This asymmetry indicates the default sits on the update-starved side of the batch-size/update-count trade-off, consistent with the critical-batch-size picture of McCandlish et al. (2018) and the large-scale measurements of Shalue et al. (2019).

The improvement does not continue: 2^{17} (~ 3760 updates) recovers only part of the gain (0.9863, 3-seed) and 2^{16} regresses past the warm baseline (1.0007), consistent with per-step gradient noise — and, at $2^{17}\text{--}2^{16}$,

reduced per-step efficiency from smaller device batches — overtaking the benefit of more updates. The optimum at this frame is 2^{18} , which we adopt as the new reference. The confidence is held below 0.9 because the sub- 2^{18} flank confounds batch size with device batch (Section 6), and because “update-count-limited” is inferred from one lever; a step-count-matched ablation (fixed steps, varied batch) has not been run.

5. Results

Table 1 summarizes all configurations from rounds 1–5. $\Delta\text{val_bpb}$ is reported against the warm 3-seed baseline mean 0.98768; negative is better. Data: campaign_log.jsonl, exp000–exp035.

Three features of the table are worth emphasizing. First, every scalar knob within the default architecture is flat at the resolution available ($\pm 1\sigma$ at $n=1$), consistent with a well-tuned upstream recipe. Second, every structural change tried so far (depth, width, attention window) is worse under this frame, because it trades throughput for capacity the 5-minute budget cannot amortize — echoing, at miniature scale, the compute-allocation logic of Hoffmann et al. (2022). Third, the batch-size column is the only one with a confirmed, gate-clearing signal, and it is non-monotonic exactly as H1 predicts.

6. Threats to Validity

1. Single-seed screening. Most negative results in Table 1 rest on one seed against $\sigma \approx 0.0015$; effects up to $\sim \pm 0.003$ could be hiding in any of them. Only the batch-size result and the matrix_lr 0.05 rejection meet the 3-seed gate. “No effect” claims should be read as “no effect resolvable at $n = 1$.”
2. Batch-size $\times$ device-batch confound below 2^{18} . The 2^{17} and 2^{16} runs also reduced the device batch (64, 32), changing kernel efficiency and per-step time, so the sub- 2^{18} regression conflates statistical (gradient noise) and systems (throughput) causes. The location of the optimum is therefore less certain than its existence.
3. Seed variance and shared hardware. $\sigma \approx 0.0015$ is itself a 3-seed estimate; the node is shared, and co-tenant load can shift step counts between waves. Concurrent controls mitigate but do not eliminate this.
4. Learning rates not re-tuned at 2^{18} . The -0.008 gain was measured at learning rates tuned (implicitly) for 2^{19} ; the true optimum batch size

could shift once learning rates are re-optimized, as batch-size/learning-rate coupling would predict (McCandlish et al., 2018).

7. Conclusion and Future Work

Under a fixed 5-minute wall-clock budget, two effects dominate the first five rounds of this campaign: a compile-cache throughput artifact that invalidates the naive first-run baseline (F1), and an update-count-limited regime in which halving the total batch size to 2^{18} tokens is the sole change to clear the pre-registered 3-seed 2σ gate, improving val_bpb by -0.008 to a mean of 0.9797 (F2). Overall confidence in the pair of findings is 0.8: F1 is mechanically verified (same code, 671 vs. ~ 1017 steps, ~ 650 vs. ~ 230 ms/step), and F2 clears its gate several times over with concurrent controls, but most null results are single-seed, and the device-batch confound below 2^{18} leaves the optimum’s exact location open.

Ranked next experiments: (1) re-tune learning rates (foremost matrix_lr, embedding_lr) at TOTAL_BATCH 2^{18} , running 3-seed paired confirmation for any pilot flashing $\geq 1\sigma$; (2) test batch-size $\times$ schedule interactions (warmup/warmdown fractions) at the doubled update count; (3) deconfound the lower flank by running 2^{17} at device batch 128 (gradient-accumulation-free) to separate gradient noise from kernel-efficiency loss; (4) structural exploration of new mechanisms (activation functions, optimizer variants, attention-window schedules), since scalar-knob tuning around the default is saturated.

References

- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., et al. Training compute-optimal large language models. In Advances in Neural Information Processing Systems 35 (NeurIPS), 2022.
- Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., and Bernstein, J. Muon: An optimizer for hidden layers in neural networks. 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- Karpathy, A. nanochat: The best ChatGPT that \$100 can buy. 2025. URL <https://github.com/karpathy/nanochat>.
- McCandlish, S., Kaplan, J., Amodei, D., and the OpenAI Dota Team. An empirical model of large-batch training. arXiv preprint arXiv:1812.06162, 2018.
- Shallue, C. J., Lee, J., Antognini, J., Sohl-Dickstein, J., Frostig, R., and Dahl, G. E. Measuring the ef-

Table 1. All configurations, rounds 1–5. $\Delta\text{val_bpb}$ is versus the warm 3-seed baseline mean 0.98768 (negative = better). Single-seed rows are screening pilots; only 3-seed rows are eligible for the 2σ keep gate.

Config	val_bpb	Δ vs. warm baseline	Seeds	Verdict
Naive baseline (cold compile, 671 steps)	1.0301	+0.0424	1	invalid
Warm baseline, TOTAL_BATCH 2^{19} (~ 1017 steps)	0.98768	—	3	reference
matrix_lr $\in \{0.03, 0.05, 0.06\}$	0.9862–0.9881	within $\pm 1\sigma$	1 (0.05: 3)	no effect
embedding_lr $\in \{0.45, 0.8\}$	0.9875–0.9877	within $\pm 1\sigma$	1	no effect
warmdown $\in \{0.4, 0.6\}$	0.9874–0.9876	within $\pm 1\sigma$	1	no effect
weight decay $\in \{0.1, 0.35\}$	0.9870–0.9878	within $\pm 1\sigma$	1	no effect
unembedding_lr 0.006 / scalar_lr 0.3	0.9864 / 0.9867	within $\pm 1\sigma$	1	no effect
Adam $\beta_1 = 0.9$	0.9904	+0.0027	1	worse, c
warmup 0.05	0.9892	+0.0015	1	worse, c
device batch 256	0.9868	~ 0 (same steps, $2\times$ VRAM)	1	no thro
depth 6 / 10 / 12	1.0222 / 0.9908 / 1.0052	+0.035 / +0.003 / +0.018	1	worse, c
aspect ratio 96 (wider)	1.0338	+0.046	1	worse, c
all-full-attention window “L”	0.9908	+0.003	1	worse, c
TOTAL_BATCH 2^{20} (506 steps)	1.0131	+0.025	1	worse, c
TOTAL_BATCH 2^{18} (~ 2020 steps)	0.9797 (best 0.9784)	-0.0080 ($\approx 4\text{--}5\sigma$)	3	keep —
TOTAL_BATCH 2^{17} , device batch 64 (~ 3760 steps)	0.9863	-0.0014	3	worse t
TOTAL_BATCH 2^{16} , device batch 32 (~ 6881 steps)	1.0007	+0.0130	1	worse, c

fects of data parallelism on neural network training.
Journal of Machine Learning Research, 20(112):1–
49, 2019.